ABAP Cheat Sheet

Standard ABAP 8.16 · ABAP Cloud 9.20 (2608) · AI-generated summary — verify against the ABAP Keyword Documentation · see the ABAP pages for details

TYPES, DATA, STRINGS

Types and data

```
TYPES ty_claim_id TYPE c LENGTH 10.
DATA  claim_id    TYPE ty_claim_id.
DATA price TYPE ty_amount VALUE '19.99'.
DATA(count) = lines( claim-items ).
DATA(first_item) = claim-items[ 1 ].
```

String templates

```
DATA(greeting) = |Hello { name }, you
  have { count } open claims.|.
{ amount DECIMALS = 2 ALIGN = RIGHT
  WIDTH = 10 }
```

Split, join, match

```
SPLIT `C-1001;Ada` AT ';' INTO TABLE
  lt_parts.
concat_lines_of( table = lt_parts
                 sep = ` | ` )
matches( val = claim_id pcre = `C-\d{4}` )
```

CONTROL FLOW

IF and CASE

```
IF amount <= 0.
  route = `rejected`.
ELSEIF amount <= 100.
  route = `auto`.
ELSE.
  route = `finance`.
ENDIF.
CASE status.
  WHEN 'N' OR 'S'.
  WHEN OTHERS.
ENDCASE.
```

Loops

```
DO 5 TIMES.
  total = total + sy-index.
ENDDO.
WHILE amount >= 1.
  amount = amount / 2.
ENDWHILE.
LOOP AT amounts INTO DATA(amount).
  CHECK amount > 0.   CONTINUE.   EXIT.
ENDLOOP.
```

INTERNAL TABLES

Change rows

```
APPEND VALUE #( claim_id = 'C-1001'
  item_no = 1 ) TO items.
MODIFY items INDEX 2 FROM VALUE #(
  amount = '125.00' ) TRANSPORTING amount.
DELETE items WHERE category = 'MEALS'.
SORT items BY category amount DESCENDING.
```

Read a row

```
DATA(second) = items[ 2 ].
IF line_exists(
  items[ claim_id = 'C-1002' ] ).
VALUE ty_item( items[ claim_id = 'X' ]
               OPTIONAL )
```

Loop, group

```
LOOP AT items ASSIGNING FIELD-SYMBOL(<item>)
  WHERE cost_center = 'CC3000'.
LOOP AT items INTO DATA(item)
  GROUP BY item-cost_center
  INTO DATA(cost_center).
```

CONSTRUCTOR EXPRESSIONS

VALUE, BASE, CONV, EXACT

```
DATA(items) = VALUE ty_items(
  ( claim_id = 'C1' item_no = 1 )
  ( claim_id = 'C1' item_no = 2 ) ).
items = VALUE #( BASE items
  ( item_no = 4 ) ).
DATA(converted) = CONV decfloat34( 1 / 5 ).
code = EXACT clen3( 'TRAVEL-EXPENSES' ).
```

COND, REDUCE

```
DATA(level) = COND string(
  WHEN total > 1000 THEN `Director`
  ELSE `Automatic` ).
REDUCE ty_item-amount(
  INIT sum = CONV ty_item-amount( 0 )
  FOR member IN GROUP cost_center
  NEXT sum = sum + member-amount )
```

ABAP SQL

Read and aggregate

```
SELECT claim_id, amount FROM zexp_claim
  WHERE amount >= @min_amount
  ORDER BY amount DESCENDING
  INTO TABLE @DATA(big).
SELECT status, COUNT( * ) AS claims,
       SUM( amount ) AS total
  FROM zexp_claim GROUP BY status
  INTO TABLE @DATA(per_status).
```

Join and write

```
SELECT c~claim_id, i~item_no
  FROM zexp_claim AS c
  LEFT OUTER JOIN zexp_item AS i
    ON i~claim_id = c~claim_id
  INTO TABLE @DATA(outer_rows).
INSERT zexp_item FROM TABLE @items.
DELETE FROM zexp_item
  WHERE claim_id = 'C-1002'.
```

CLASSES AND EXCEPTIONS

Class, interface, inheritance

```
CLASS zcl_exp_claim_validator DEFINITION
  PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS is_valid
      IMPORTING amount TYPE ty_amount
      RETURNING VALUE(result)
        TYPE abap_bool.
ENDCLASS.
CLASS lcl_full_payout DEFINITION
  INHERITING FROM lcl_payout_base.
METHODS lif_payout~calculate REDEFINITION.
```

Exceptions

```
CLASS zcx_exp_invalid_amount DEFINITION
  PUBLIC INHERITING FROM cx_static_check
  FINAL CREATE PUBLIC.
RAISE EXCEPTION NEW zcx_exp_invalid_amount(
  amount = amount ).
TRY.
  CATCH zcx_exp_invalid_amount INTO
    DATA(invalid).
ENDTRY.
```

ABAP UNIT

Test class and asserts

```
CLASS ltcl_limit_check DEFINITION FINAL
  FOR TESTING DURATION SHORT
  RISK LEVEL HARMLESS.
  PRIVATE SECTION.
    METHODS setup.
    METHODS rejects_amount_above_limit
      FOR TESTING.
ENDCLASS.
cl_abap_unit_assert=>assert_equals(
  act = limit_double->calls exp = 1 ).
cl_abap_unit_assert=>fail( msg = 'text' ).
```

CDS VIEW ENTITIES

View entity and association

```
define view entity ZI_ExpClaim
  as select from zexp_claim
  association of many to one
    ZI_ExpEmployee as _Employee
    on $projection.EmployeeId =
       _Employee.EmployeeId
{
  key claim_id    as ClaimId,
      employee_id as EmployeeId,
      _Employee.FullName as EmployeeName
}
```

RAP, EML, TOOLS

RAP (EML needs an SAP system)

```
managed implementation in class
  zbp_i_expclaim unique;
define behavior for ZI_ExpClaim alias Claim
persistent table zexp_claim
define service ZSD_ExpClaim {
  expose ZC_ExpClaim as Claim;
}
READ ENTITIES OF zi_expclaim ...
COMMIT ENTITIES.
```

ADT, abaplint, ATC

```
Ctrl+F2  check       Ctrl+F3  activate
Shift+F1  format     F1  keyword docs
npx abaplint lint-demo/abaplint.json
DATA text TYPE string ##needed.
```